



UPPSALA
UNIVERSITET

UPTEC IT 26021

Examensarbete 30 hp

Juni 2026

Optimising Weak Reference Processing in HotSpot's Z Garbage Collector

Reference Pipeline Optimisations and an Exploratory
Weak-Field Mechanism for HotSpot

Fredrik Hammarberg



Abstract

In Java, weak references are a non-trivial source of garbage collection overhead in workloads where objects frequently become weakly reachable. In the Z Garbage Collector (ZGC), all discovered weak references follow the same processing pipeline regardless of whether they are associated with a ReferenceQueue. For references that are never registered with a queue this leads to unnecessary pending-list and enqueue-path work. This thesis investigates whether targeted modifications to ZGC's reference-processing pipeline can reduce this overhead, and whether representing weak semantics directly in object fields rather than through separate WeakReference objects can reduce overhead further.

Three orthogonal pipeline optimisations for queue-less WeakReference processing were implemented in an OpenJDK research fork: a skip-enqueue separation that routes queue-less weak references to a separate discovered list, bypassing the pending list entirely, a dynamic-array representation that replaces the intrusive linked list used during reference discovery to improve traversal locality, and a specialised clear path that removes unnecessary per-reference CAS operations. In addition, an exploratory weak-field mechanism was implemented through a Java field annotation and corresponding ZGC VM support, enabling a representation-level comparison under the same experimental setup.

The implementations were evaluated using synthetic microbenchmarks executed on a supercomputer cluster with 250 repeated runs per variant. Two benchmark designs were used: a single-object stress benchmark that concentrates the entire weak-reference load into one GC cycle, and a multi-object benchmark that releases references gradually across five GC cycles.

The results show that combining the specialised clear path with the dynamic array is the most effective pipeline configuration. In the single-object benchmark, this combination reduces median non-strong reference-processing time by 81% and median major-collection time by 8%. In the multi-object benchmark, the non-strong subphase reduction is 57%, while the end-to-end major-collection gain is 1.1%. Dynamic-array variants incur substantially higher auxiliary garbage collector (GCr) memory than the linked-list baseline. The combination of all three optimisations together, skip-enqueue separation, dynamic array, and specialised clear path, offers the best balance: it matches the same 81% reduction in non-strong processing time whilst consuming approximately 30% less auxiliary GCr memory than the clear-path-plus-array pair alone.

The weak-field mechanism achieves substantially lower major-collection times than all WeakReference pipeline variants, with reductions of 41% in the single-object benchmark and 28% in the multi-object benchmark relative to the baseline. Its advantage is structural: by eliminating WeakReference wrapper objects from the heap entirely, it reduces the total volume of GC work across all phases of the collection cycle, not only during reference processing. This comes at a significant implementation cost: the weak-field prototype spans 27 files across the compiler, class-file parser, interpreter, JIT compilers, JVMTI, and GCr subsystems.

Together, the results suggest that, within the evaluation setting of queue-less synthetic workloads on a single hardware platform, the choice of how weak semantics is represented in the language has a larger impact on total GC cost than pipeline-level optimisations. These findings indicate that meaningful improvements in workloads of this kind may require reconsidering how weak semantics is encoded in the language rather than only refining the processing pipeline, though how this conclusion generalises to production workloads with diverse reference patterns and multiple hardware platforms remains an open question.

Teknisk-naturvetenskapliga fakulteten

Uppsala universitet, Utgivningsort Uppsala

Handledare: Stefan Johansson Ämnesgranskare: Tobias Wrigstad

Examinator: Anders Arweström Jansson



Sammanfattning

I Java utgör svaga referenser en icke-trivial källa till overhead vid skräpinsamling i system där objekt frekvent blir svagt nåbara. I Z Garbage Collector (ZGC) hanteras alla identifierade svaga referenser i samma pipeline oavsett om de är associerade med en *ReferenceQueue* eller inte. För referenser som aldrig registreras med en kö leder detta till onödigt arbete i en pending list och sedan köläggning. Detta arbete undersöker om riktade modifieringar av ZGC:s referenspipeline kan minska denna overhead, och om representationen av svag semantik direkt i objektfält snarare än genom separata *WeakReference*-objekt kan minska overheaden ytterligare.

Tre ortogonala pipelineoptimeringar för hantering av köfria *WeakReference* implementerades i en OpenJDK-fork: separata listor som kringgår köläggningen för köfria svaga referenser, en representation baserad på en dynamisk array som ersätter den intrusiva länkade listan som används vid referensidentifiering för att förbättra traverseringslokalitet, och en specialiserad rensningsfunktion som eliminerar onödiga CAS-operationer för varje referens. Dessutom implementerades en explorativ svagfältmekanism via en Java-fältannotering och tillhörande ZGC VM-stöd, vilket möjliggör en representationsnivåjämförelse under samma experimentella upplägg.

Implementationerna utvärderades med syntetiska mikrobenchmarks körda på ett superdatorcluster med 250 upprepade körningar per variant. Två benchmarkupplägg användes: ett enkel-objekt-stressbenchmark som koncentrerar hela svagreferensbelastningen till en enda GC-cykel, och ett fler-objekt-benchmark som frigör referenser gradvis över fem GC-cykler.

Resultaten visar att kombinationen av den specialiserade rensningsfunktionen med den dynamiska arrayen är den mest effektiva pipelinekonfigurationen. I enkel-objekt-benchmarket minskar denna kombination mediantiden för icke-stark referenshantering med 81% och median major-insamlingstiden med 8%. I fler-objekt-benchmarket är minskningen i den icke-starka underfasen 57%, medan end-to-end-vinsten för major-insamlingstiden är 1.1%. Varianterna med dynamisk array medför avsevärt högre extra GC-minne än basfallet med länkad lista. Kombinationen av alla tre optimeringarna, *skip-enqueue*-separering, dynamisk array och den specialiserade *clear path*-metoden, ger den bästa kombinationen. Den uppnår samma minskning på 81% av bearbetningstiden för icke-starka referenser, samtidigt som den använder ungefär 30% mindre extraminne än kombinationen av enbart *clear path* och dynamisk array.

Svagfält-mekanismen uppnår avsevärt kortare major-insamlingstider än alla *WeakReference*-pipelinevarianter, med minskningar på 41% i enkel-objekt-benchmarket och 28% i fler-objekt-benchmarket relativt basfallet. Fördelen är strukturell: genom att eliminera *WeakReference*-omslagsobjekt från heapen helt minskas den totala volymen GC-arbete över alla faser i insamlingscykeln, inte enbart under referenshantering. Detta medför dock en betydande implementeringskostnad: svagfält-prototypen spänner över 27 filer i kompilatorn, klassfilsparsern, tolken, JIT-kompilatorer, JVMTI och GC-delsystem.

Sammantaget antyder resultaten att, inom det utvärderade scenariot med köfria syntetiska arbetsbelastningar på en och samma hårdvaruplattform, har valet av hur svag semantik representeras i språket en större inverkan på den totala GC-kostnaden än optimeringar på pipelinenivå. Dessa resultat tyder på att meningsfulla förbättringar i sådana arbetsbelastningar kan kräva en omvärdering av hur svag semantik kodas i språket snarare än enbart en förfining av hanteringspipelinen, men huruvida detta slutsats generaliserar till produktionsarbetsbelastningar med varierande referensmönster och olika hårdvaruplattformar förblir en öppen fråga.

Teknisk-naturvetenskapliga fakulteten

Uppsala universitet, Utgivningsort Uppsala

Handledare: Stefan Johansson Ämnesgranskare: Tobias Wrigstad

Examinator: Anders Arweström Jansson